

Improper Platform Usage & Extraneous Functionality

Introduction

What is Improper Platform Usage and Extraneous Functionality?

According to research on mobile application security:

Improper platform usage in mobile applications often arises from a combination of oversight, misunderstanding, or a lack of adherence to platform-specific guidelines. Extraneous functionality in mobile penetration testing refers to unnecessary or unintended features, services, or code within an application that can introduce security risks. These features may not be part of the core functionality but still exist due to oversight, incomplete development, or poor security practices.

Vulnerabilities and Root Causes

- **Misuse of Platform Features**

- Violation of published guidelines or common best practices, as well as the unintentional misuse of specific platform features, leads to improper platform usage.

- **Development Oversight**

- The inclusion of debug mode, backup code being enabled, or commenting about sensitive data in source code often results in extraneous functionality.

What is the impact of these issues?

- **Unauthorized Access and Data Disclosure**

- Poorly managed activities risk unauthorized access to sensitive functions, while inadequate log analysis can expose sensitive data leading to breaches.

- **Exposure of Credentials and Security Keys**

- Storing keys insecurely or including old code with known vulnerabilities can expose user credentials, API keys, and personal data to attackers.

Case Study

Bypassing Authentication via Exported Activities

An application designed for enterprise use misconfigures its primary administrative activity as "exported=true" in the Android Manifest, violating standard security conventions.

The Vulnerability (Unrestricted Activity Access)

The developer intends for the administrative panel to be accessible only after a successful login. However, because the activity is exported, it can be launched directly by any other application on the device or via command-line tools without going through the login screen.

The Attack

An attacker uses ADB to identify the exported administrative activity and triggers it using a simple intent command: `adb shell am start -n com.app.package/.AdminActivity`. This bypasses all authentication checks implemented in the standard user workflow.

Result

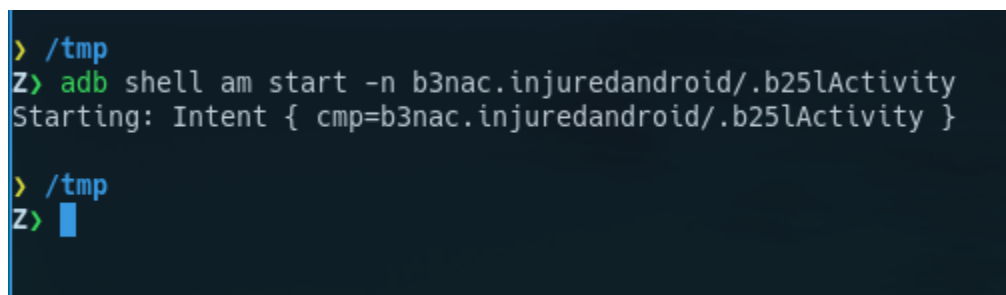
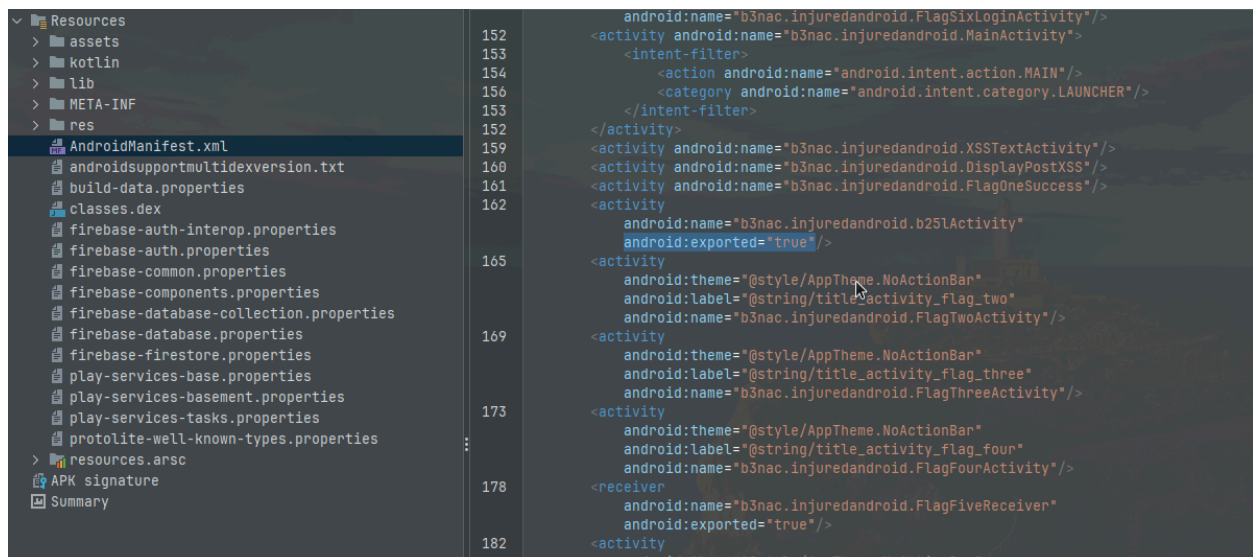
The attacker successfully gains access to internal administrative functions, demonstrating how improper platform usage regarding activity management can completely compromise application security.

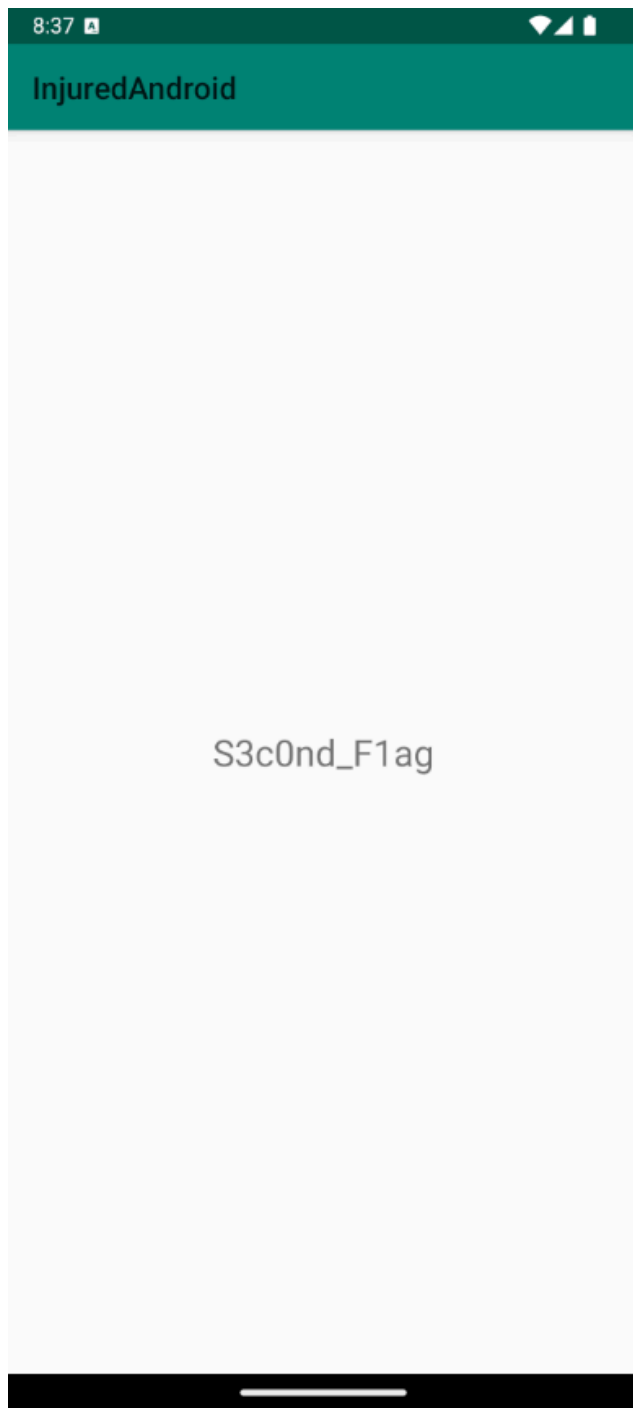
Practical (InjuredAndroid)

To find the second flag, we first decompile the app via jadx and then start looking for clues in AndroidManifest.xml

In AndroidManifest.xml we can look for the string **android:export="true"** next to several activities.

Android export set to true means that this activity can be called externally via adb. This is an improper platform usage vulnerability.





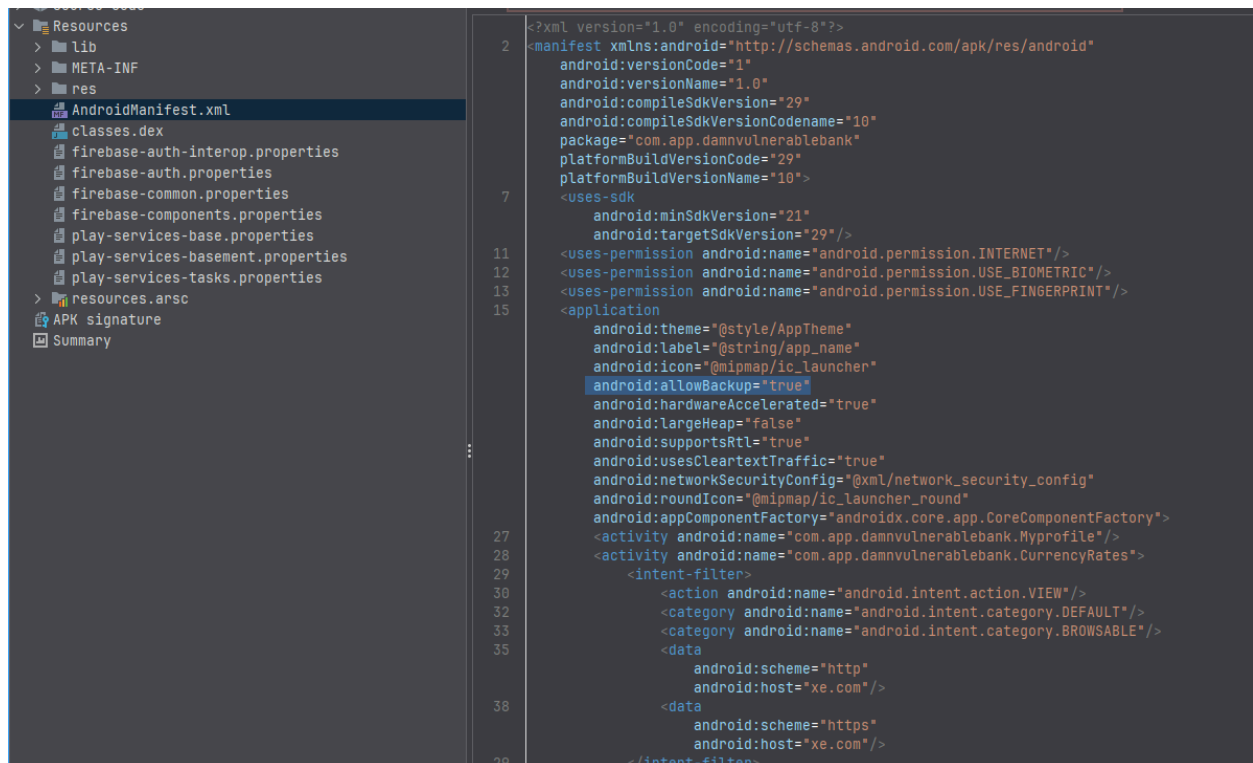
This way we successfully capture the second flag.

S3c0nd_F1ag

Practical (Damn Vulnerable Bank)

Goal: Check if the app is debuggable and backup is enabled

To find out if the app is debuggable and backup is enabled, we have to decompile the app via jadx and check AndroidManifest.xml file.



In AndroidManifest.xml, <application> tag we see that **android:allowBackup="true"** is found which means we can use the **\$ adb backup** command to backup app data.

For a **banking app** this is both **Improper Platform Usage** and an **Extraneous Functionality**.

android:debuggable="true" is not found in the <application> tag which means debugging is not possible.

Mitigations

- **Follow Platform Guidelines**

- Strictly adhere to Android and iOS security documentation to avoid misconfiguring activities and intents.

- **Disable Production Debugging**

- Ensure that debuggable flags are set to false and all developer-only code is removed before release.

- **Code Sanitization**

- Remove sensitive comments, hardcoded strings, and diagnostic logging from the source code.

Conclusion

Improper Platform Usage and Extraneous Functionality represent significant security risks that arise during the development and configuration phases of mobile applications. By failing to follow platform best practices or leaving unnecessary code in production builds, developers inadvertently create entry points for attackers. Mitigating these risks requires a proactive approach centered on strict adherence to platform security guidelines, comprehensive code sanitization, and the disabling of all non-essential features before deployment.

References

<https://github.com/B3nac/InjuredAndroid>

<https://github.com/rewanthtammana/Damn-Vulnerable-Bank>

<https://owasp.org/www-project-mobile-top-10>

<https://owasp.org/www-project-mobile-top-10/2016-risks/m1-improper-platform-usage>

<https://medium.com/@abjurato/owasp-for-ios-m1-improper-platform-usage-part-1-7aff742c50ee>

<https://owasp.org/www-project-mobile-top-10/2016-risks/m10-extraneous-functionality>